



Sunbeam Institute of Information Technology
Pune and Karad

Data structures and Algorithms

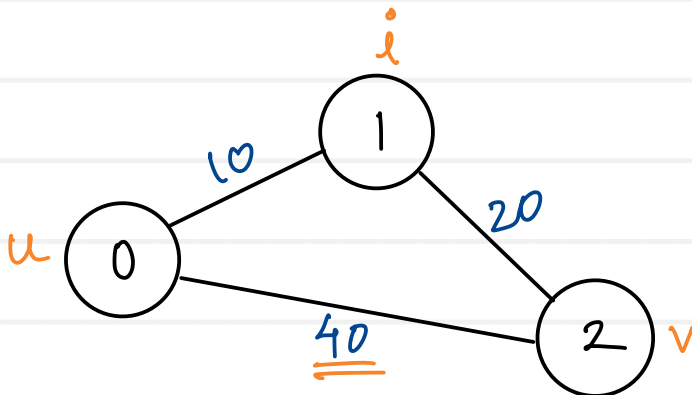
Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

1. Create distance matrix to keep distance of every vertex from each vertex.
Initially assign it with weights of all edges among vertices
(i.e. adjacency matrix).

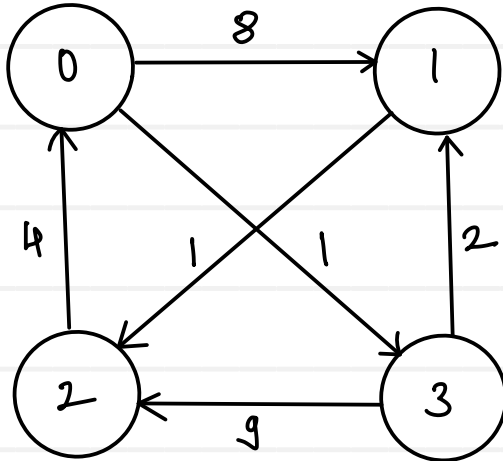
} dp Array

2. Consider each vertex (i) in between pair of any two vertices (u, v) and
find the optimal distance between u & v considering intermediate vertex
i.e. $\text{dist}(u, v) = \text{dist}(u, i) + \text{dist}(i, v)$,
if $\text{dist}(u, i) + \text{dist}(i, v) < \text{dist}(u, v)$.



if ($\text{dist}[u][i] + \text{dist}[i][v] < \text{dist}[u][v]$)
 $\text{dist}[u][v] = \text{dist}[u][i] + \text{dist}[i][v];$

Floyd Warshall Algorithm



$$d = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} \infty & 8 & \infty & 1 \\ \infty & \infty & 1 & \infty \\ 4 & \infty & \infty & \infty \\ \infty & 2 & 9 & \infty \end{bmatrix} \end{matrix}$$

$$d = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ 4 & \infty & 0 & \infty \\ \infty & 2 & 9 & 0 \end{bmatrix} \end{matrix}$$

$$d_0 = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 9 & 0 \end{bmatrix} \end{matrix}$$

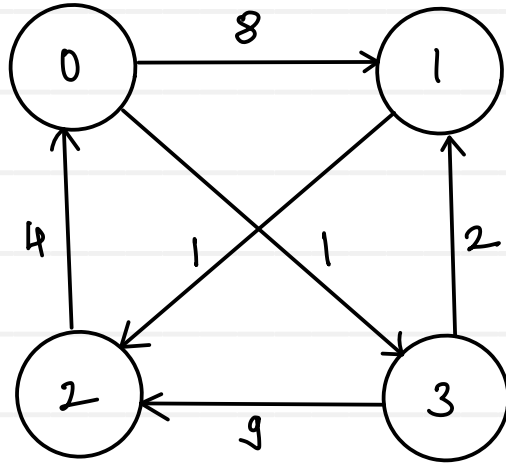
$$d_0 = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 9 & 0 \end{bmatrix} \end{matrix}$$

$$d_1 = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 8 & 9 & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 3 & 0 \end{bmatrix} \end{matrix}$$

$$d_2 = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 8 & 9 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 12 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix} \end{matrix}$$

$$d_3 = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 3 & 4 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 7 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix} \end{matrix}$$

Floyd Warshall Algorithm



$$\text{itrs} = v * v * v$$

$$\text{Time} \propto v^3$$

$$T(v) = O(v^3)$$

$$S(v) = O(v^2)$$

```

int dist[vCount][vCount];
for(int u=0; u<vCount; u++){
    for(int v=0; v<vCount; v++){
        dist[u][v] = adjMatrix[u][v];
    }
    dist[u][u] = 0;
}

```

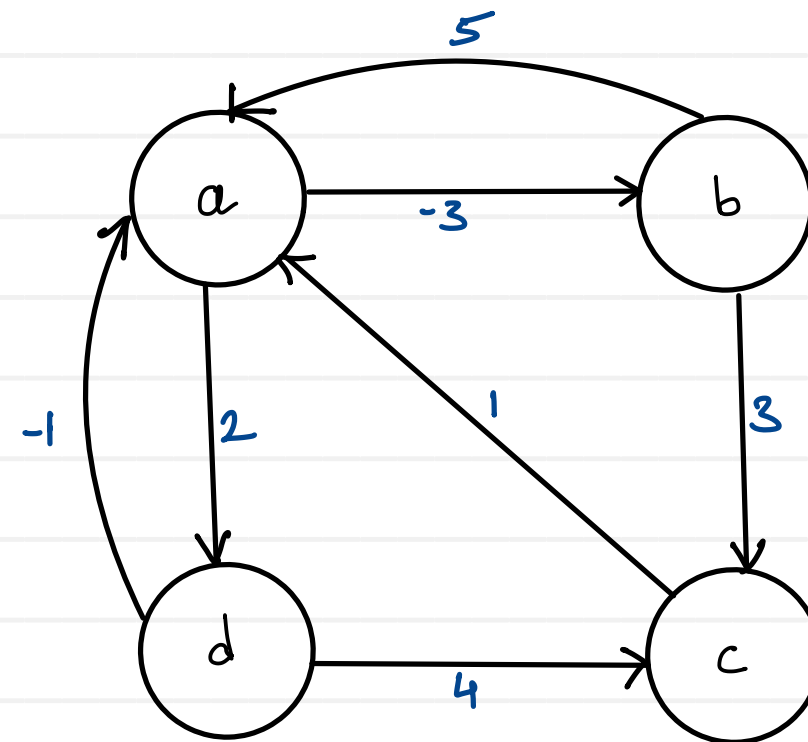
```

v times → for(int i=0; i<vCount; i++){
    v times → for(int u=0; u<vCount; u++){
        v times → for(int v=0; v<vCount; v++){
            if(dist[u][i] + dist[i][v] < dist[u][v])
                dist[u][v] = dist[u][i] + dist[i][v];
        }
    }
}

```

Johnson's Algorithm

- Time complexity of Floyd Warshall is $O(V^3)$.
- Applying Dijkstra's algorithm on V vertices will cause time complexity $O(V * V \log V)$. This is faster than Floyd Warshall.
- However Dijkstra's algorithm can't work with negative edges.
- Johnson uses Bellman ford to re weight all edges in graph to remove negative edges. Then apply Dijkstra's algorithm to all vertices to calculate shortest distance.
- Finally re weight distance to consider original edge weights.
- Time complexity of the algorithm:
 $O(\underbrace{VE}_{\text{Bellman ford}} + \underbrace{V^2 \log V}_{\text{Dijkstra's}})$



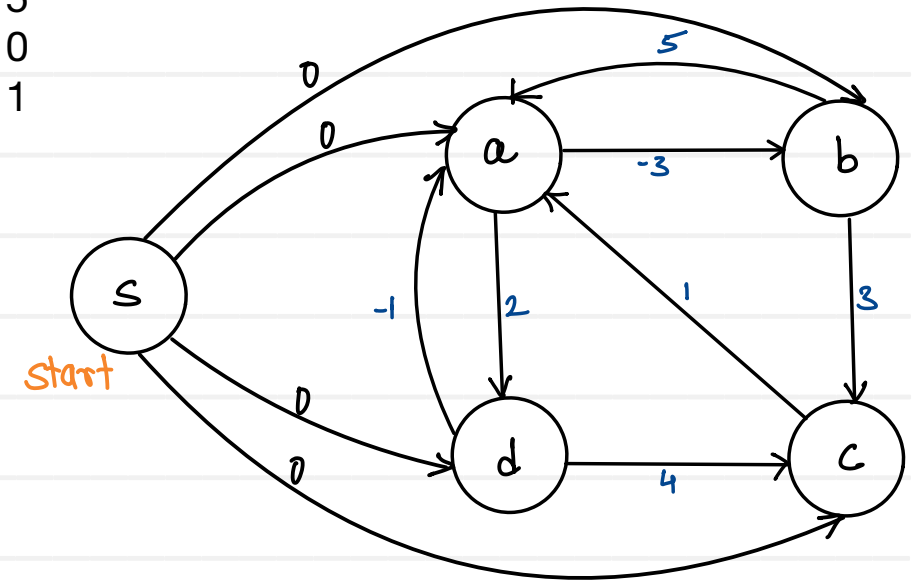
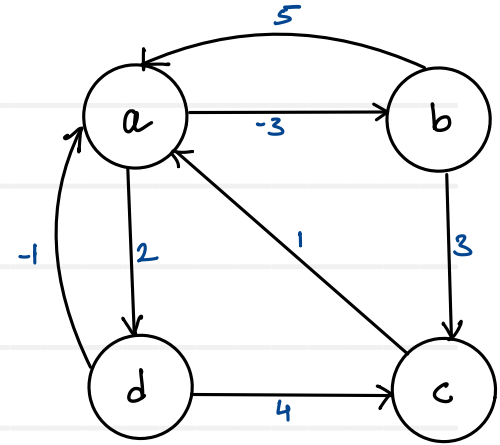
Bellman
ford

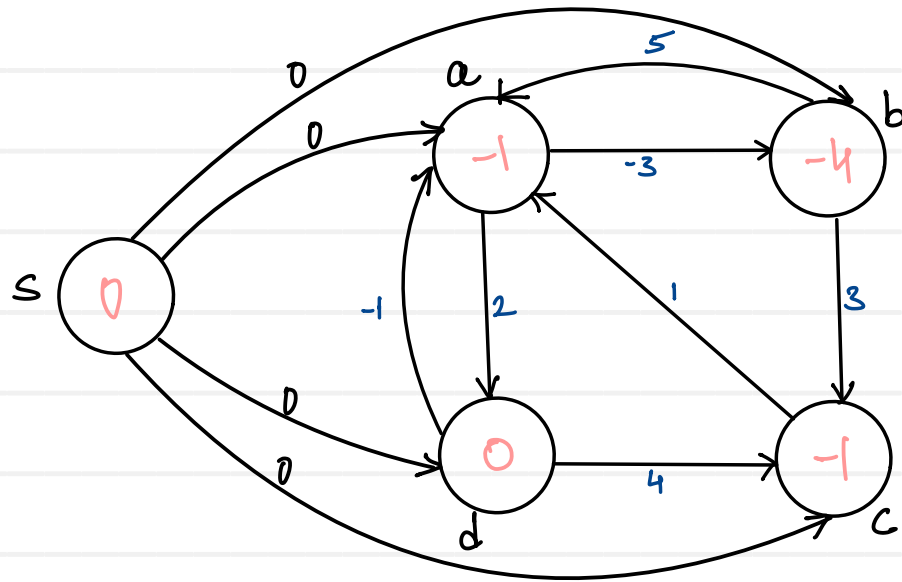
Dijkstra's

Johnson's Algorithm

- Add a vertex (s) into a graph and add edges from it all other vertices, with weight 0.
- Find shortest distance of all vertices from (s) using Bellman Ford algorithm.
 $a = -1, b = -4, c = -1, d = 0$ and $s = 0$
- Re weight all edges (u, v) in the graph, so that they become non negative.
 $\text{weight}(u,v) = \text{weight}(u,v) + d(u) - d(v)$
- Apply Dijkstra's algorithm on every vertex of graph and find distances.
- Re weight all distances against original graph
 $\text{dist}(u,v) = \text{dist}(u,v) + d(v) - d(u)$

$$\begin{aligned} w(a, b) &= 0 \\ w(b, a) &= 2 \\ w(b, c) &= 0 \\ w(c, a) &= 1 \\ w(d, c) &= 5 \\ w(d, a) &= 0 \\ w(a, d) &= 1 \end{aligned}$$





$$\text{weight}(u-v) = \text{weight}(u-v) + \text{dist}[u] - \text{dist}[v]$$

Edges

$a \rightarrow b$

$$\text{wt}(a,b) = -3 + -1 - -4 = -4 + 4 = 0$$

$b \rightarrow a$

$$\text{wt}(b,a) = 5 + -4 - -1 = 6 - 4 = 2$$

$b \rightarrow c$

$$\text{wt}(b,c) = 3 + -4 - -1 = 4 - 4 = 0$$

$c \rightarrow a$

$$\text{wt}(c,a) = 1 + -1 - -1 = 1$$

$d \rightarrow c$

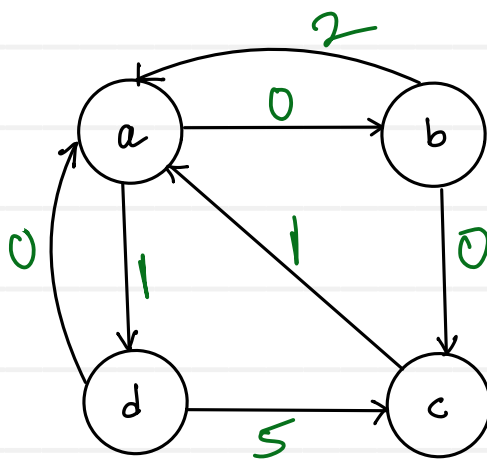
$$\text{wt}(d,c) = 4 + 0 - -1 = 5$$

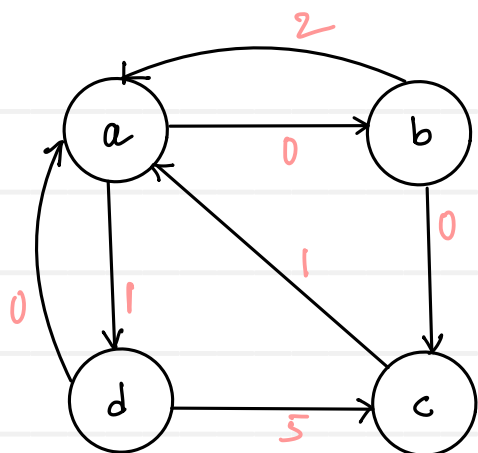
$d \rightarrow a$

$$\text{wt}(d,a) = -1 + 0 - -1 = 0$$

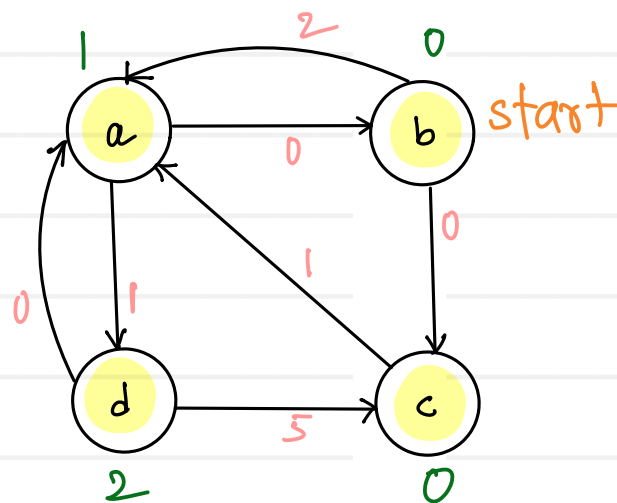
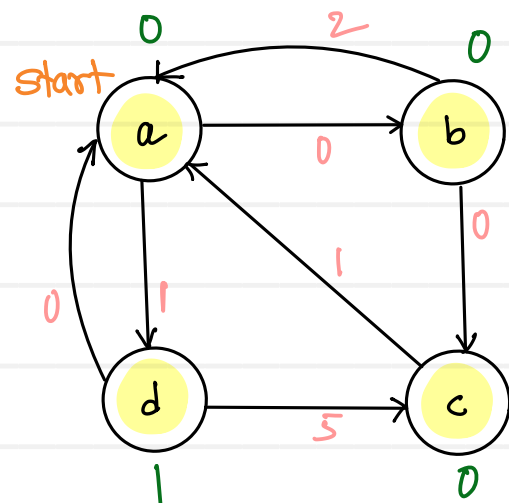
$a \rightarrow d$

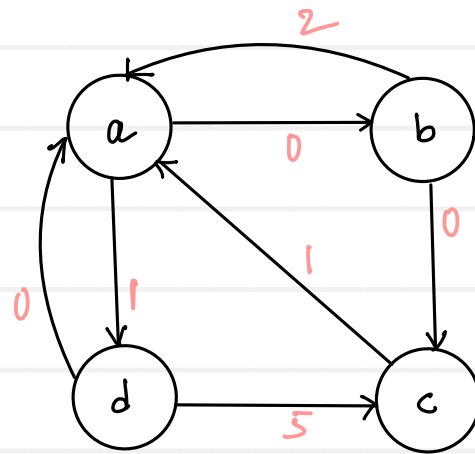
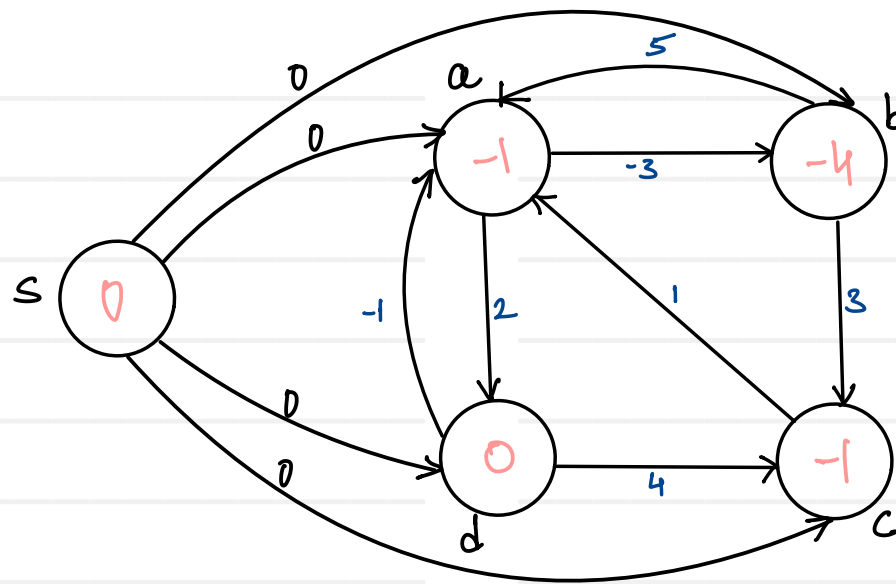
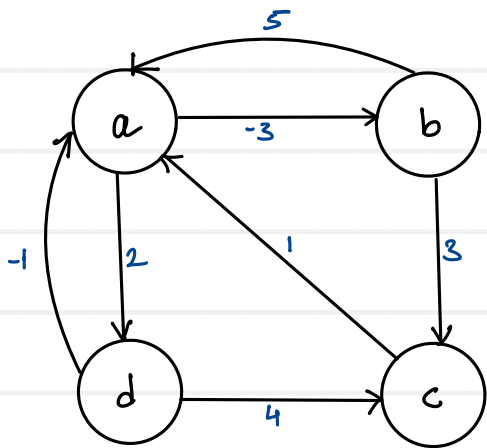
$$\text{wt}(a,d) = 2 + -1 - 0 = 1$$





	a	b	c	d
a	0	0	0	1
b	1	0	0	2
c				
d				





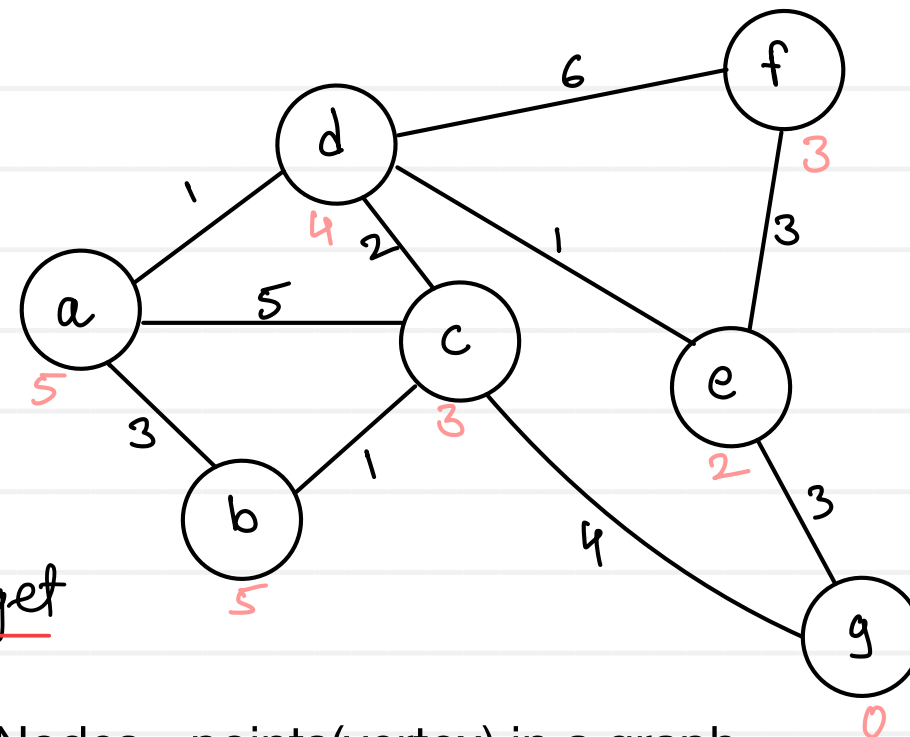
	a	b	c	d
a	0	0	0	1
b	1	0	0	2
c				
d				

$$\text{dist}(u, v) = \text{dist}(u, v) + d(v) - d(u)$$

$$\begin{aligned} \text{dist}(a, a) &= 0 + -1 - -1 = 0 \\ \text{dist}(a, b) &= 0 + -4 - -1 = -3 \\ \text{dist}(a, c) &= 0 + -1 - -1 = 0 \\ \text{dist}(a, d) &= 1 + 0 - -1 = 2 \\ \text{dist}(b, a) &= 1 + -1 - -4 = 4 \\ \text{dist}(b, b) &= 0 + -4 - -4 = 0 \\ \text{dist}(b, c) &= 0 + -1 - -4 = 3 \\ \text{dist}(b, d) &= 2 + 0 - -4 = 6 \end{aligned}$$

	a	b	c	d
a	0	-3	0	2
b	4	0	3	6
c				
d				

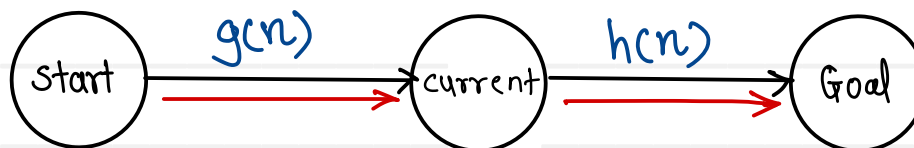
- Popular technique used in path finding and graph traversals.
- A* algorithm combines
 1. Dijkstra's algorithm
 2. Greedy Breadth First Approach
- It considers both: *(informed search)*
 - distance already traveled from the start
 - estimate of the remaining distance to the goal / target
- This combination helps A* to make informed decisions about which path to explore next.
- This makes A* both efficient and accurate



- Nodes - points(vertex) in a graph
- Edges - links/connections between nodes
- Path cost - actual cost of moving from one node to another
- Heuristic - estimated cost from any node to the goal

- Below three functions work together to guide the search process towards the most promising paths
 - $f(n)$ - total estimated cost
 - $g(n)$ - path cost / traveled distance from start to current node
 - $h(n)$ - estimated cost from current to goal / target

$$f(n) = g(n) + h(n)$$



$g(n)$: known distance from start to current node

$$g(n_k) = \sum_{i=0}^k w(n_i)$$

$h(n)$: heuristic function
- estimated cost from current to goal vertex.

- common heuristic functions:

coordinates of current node - (x_1, y_1)
coordinates of goal node - (x_2, y_2)

1. Manhattan distance

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

2. Diagonal distance

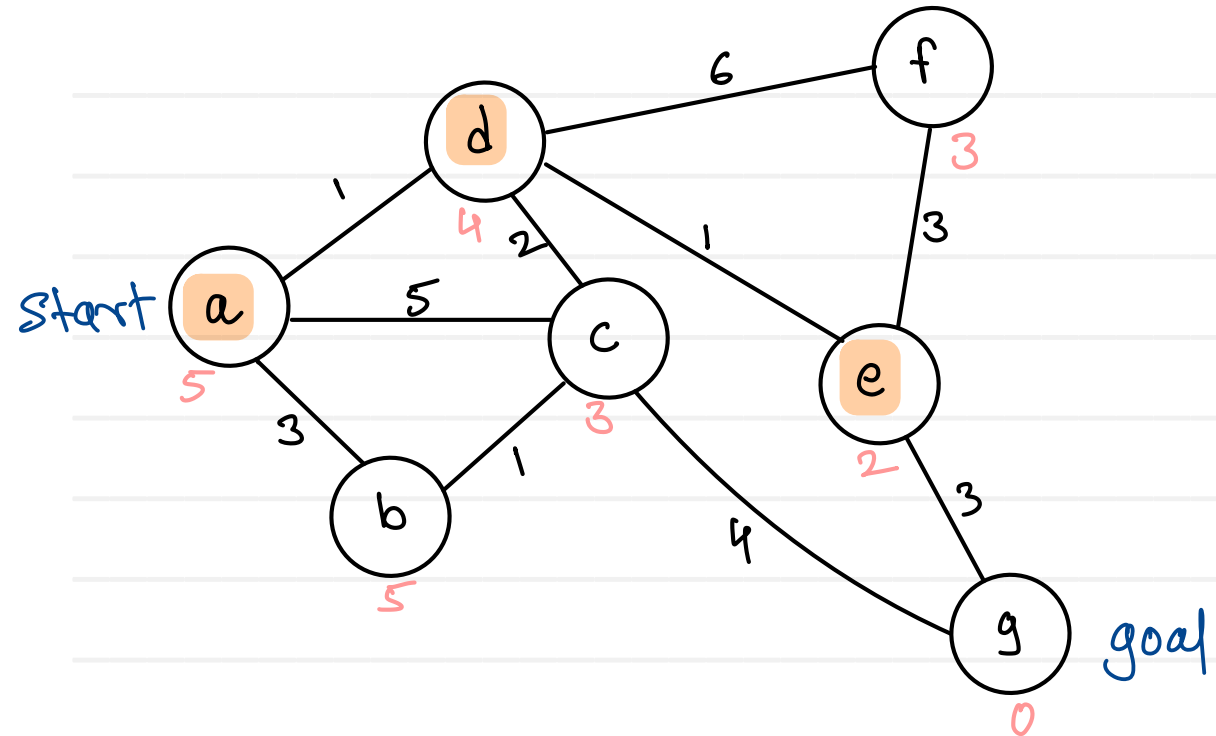
$$h(n) = \text{MAX}(|x_1 - x_2|, |y_1 - y_2|)$$

3. Euclidean distance

$$h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$



A* Algorithm



	$g(n)$	$h(n)$	$f(n)$
a	0	5	5
b	3	5	8
c	5	3	8
a d	1	4	5
c	1+2	3	6
a d e	1+1	2	4
f	1+6	3	10
f	1+1+5	3	10
a d e g	1+1+3	0	5

Graph applications

- Graph represents flow of computation/tasks. It is used for resource planning and scheduling. MST algorithms are used for resource conservation. DAG are used for scheduling in Spark or Tez.
- In OS, process and resources are treated as vertices and their usage is treated as edges. This resource allocation algorithm is used to detect deadlock.
- In social networking sites, each person is a vertex and their connection is an edge. In Facebook person search or friend suggestion algorithms use graph concepts.
- In world wide web, web pages are like vertices; while links represents edges. This concept can be used at multiple places.
 - Making sitemap
 - Downloading website or resources
 - Developing web crawlers
 - Google page-rank algorithm
- Maps uses graphs for showing routes and finding shortest paths. Intersection of two (or more) roads is considered as vertex and the road connecting two vertices is considered to be an edge.

Merge Sorted Array

Given two sorted integer arrays `nums1` and `nums2` in ascending order, and two integers `m` and `n` representing the number of elements in `nums1` and `nums2` respectively.

Merge `nums1` and `nums2` into a single array sorted in ascending order.

The final sorted array should be stored inside the array `nums1`. To accommodate this, `nums1` has a length of `m + n`.

Example 1:

Input: `nums1` = [1,2,3,0,0,0], `m` = 3, `nums2` = [2,5,6], `n` = 3

Output: [1,2,2,3,5,6]

Example 2:

Input: `nums1` = [1], `m` = 1, `nums2` = [], `n` = 0

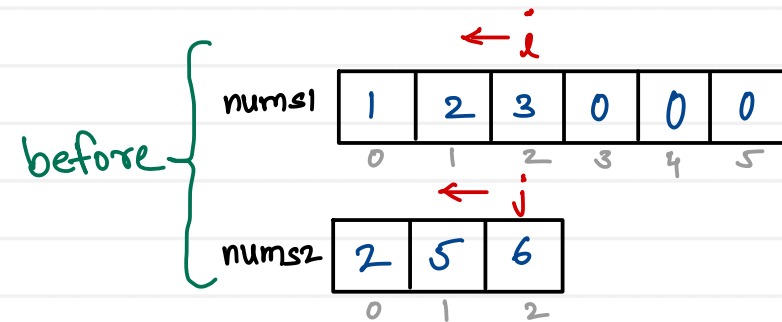
Output: [1]

Example 3:

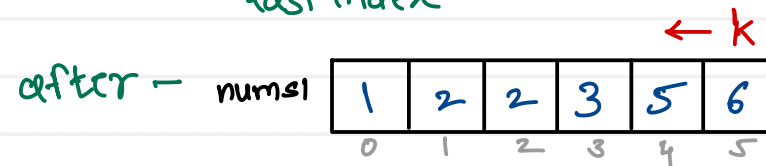
Input: `nums1` = [0], `m` = 0, `nums2` = [1], `n` = 1

Output: [1]

As sorted array to be stored in `nums1` again, need to start from last elements of the arrays.



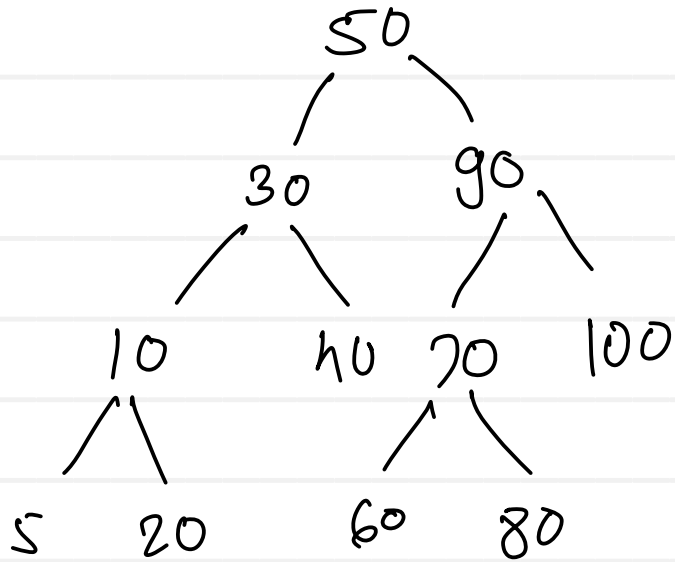
compare i th & j th element, whichever is maximum put it into `nums1` from last index



Time complexity:

$$T(n) = O(n)$$

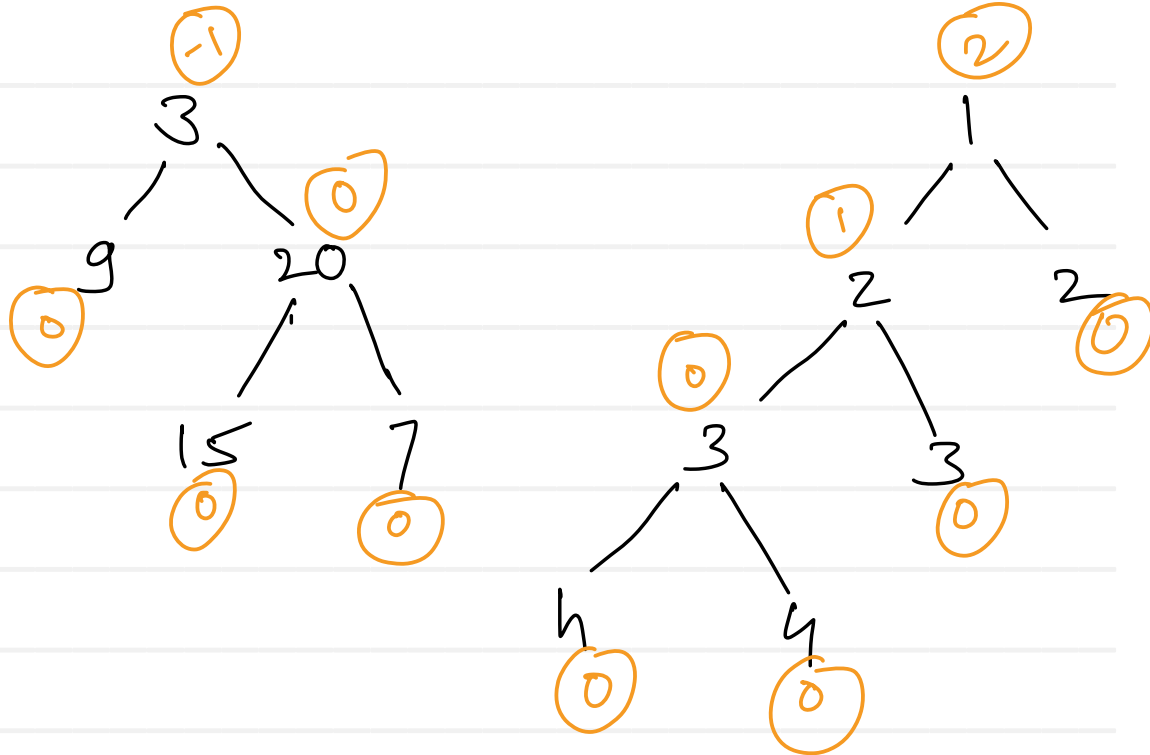
Strictly binary tree or not



traverse each node in tree using BFS
if node has single child,
return false

return true - when all nodes are
traversed

Balanced binary tree or not



```

int height(Node trav) {
    if(trav == null) return 0;
    int hl = height(trav.left);
    int hr = height(trav.right);

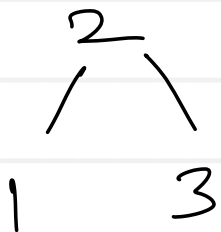
    if(hl == -1 || hr == -1)
        return -1;

    if(Math.abs(hl - hr) > 1)
        return -1;

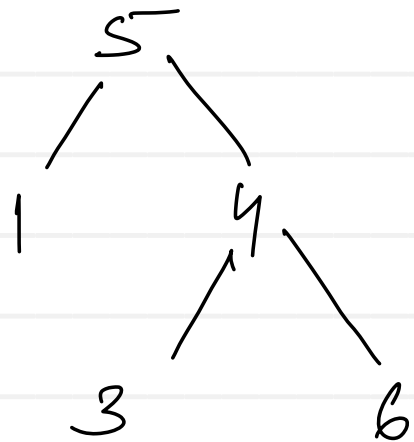
    int max = hl > hr ? hl : hr;
    return max + 1;
}

boolean isBalanced(root) {
    return height(root) != -1;
}
    
```

Validate binary search tree



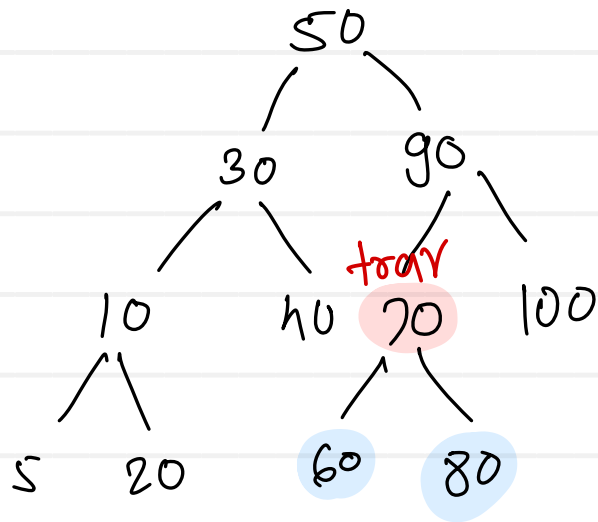
trav	left	Right
2	1	3
1	null	null
3	null	null



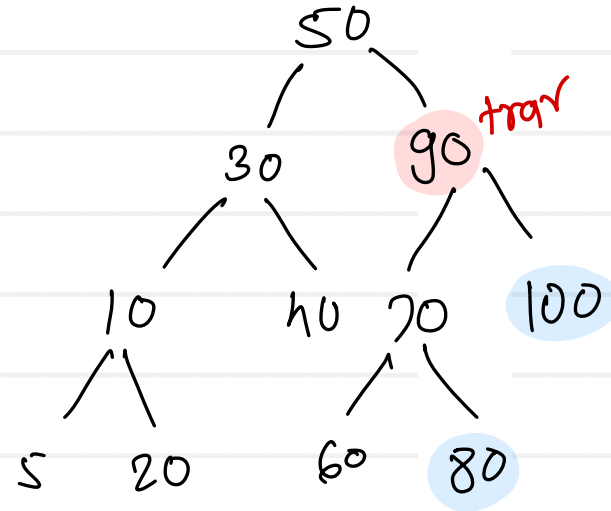
trav	left	right
5	1	4
1		
4	3	6

4 X

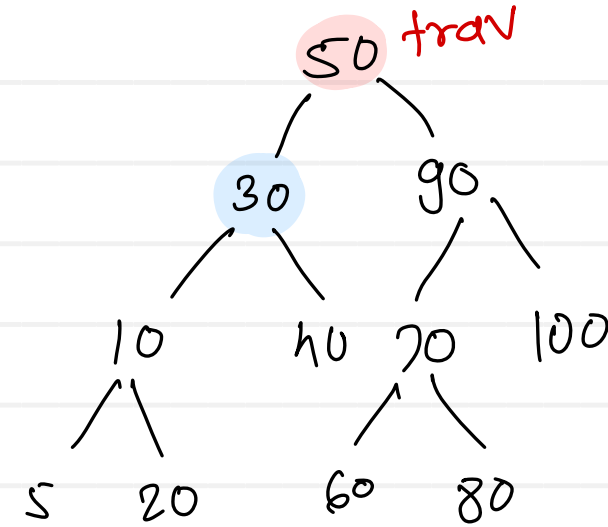
Nearest common ancestor



60, 80 → 70

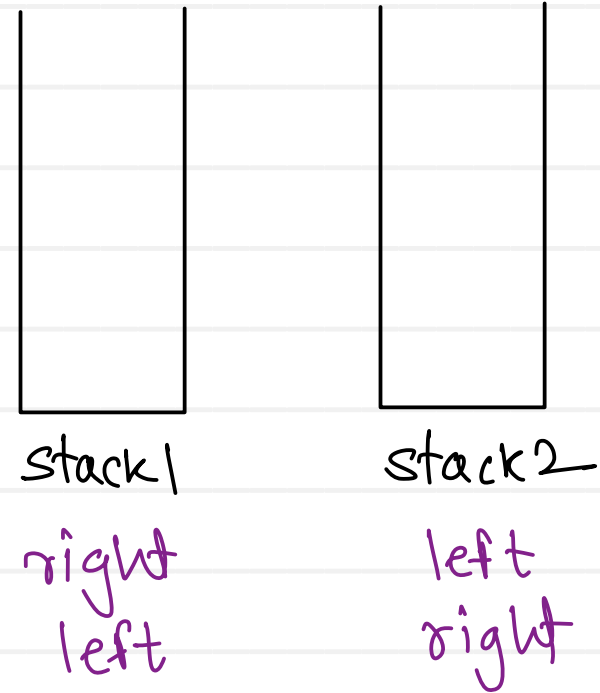
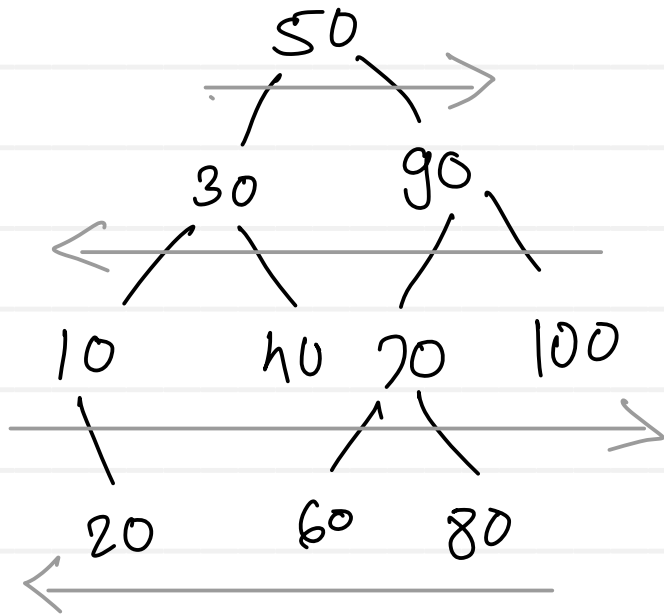


80, 100 → 90



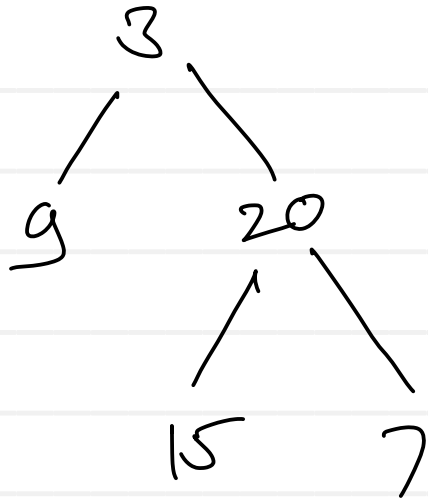
30, 75 → ~~75~~
~~X~~

Binary tree zigzag level order traversal (spiral order)



Output: [50] [90, 30] [10, 40, 70, 100] [80, 60, 20]

Minimum depth of binary tree

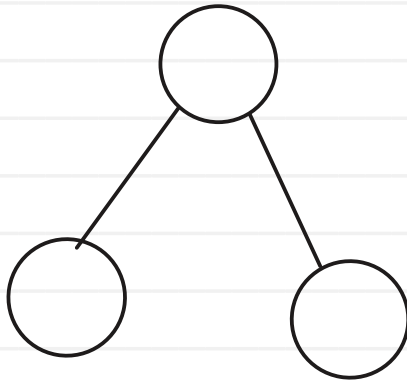


Kth Largest Element in an Array

- Given an integer array `nums` and an integer `k`, return the `k`th largest element in the array.
- Note that it is the `k`th largest element in the sorted order, not the `k`th distinct element.
- Can you solve it without sorting?
- Example 1:
 - Input: `nums = [3,2,1,5,6,4]`, `k = 2`
 - Output: 5
- Example 2:
 - Input: `nums = [3,2,3,1,2,4,5,5,6]`, `k = 4`
 - Output: 4

```
int findKthLargestElement(int nums[], int k) {  
    PriorityQueue<Integer> q = new  
        PriorityQueue<>((x, y) -> y - x);  
    for(int num : nums)  
        q.offer(num);  
    int result = 0;  
    while (--k >= 0) {  
        result = q.poll();  
    }  
    return result;  
}
```

3 2 1 5 6 4 $k = 2$



```

int findKthLargestElement(int nums[], int k) {
    PriorityQueue<Integer> q = new PriorityQueue<>( );

    for(int num : nums) {
        q.offer(num);
        if (q.size > k)
            q.poll();
    }
    return q.peek();
}
  
```

Top K Frequent Elements

- Given an integer array nums and an integer k, return the k most frequent elements. You may return the answer in any order.

- Example 1:

- Input: nums = [1,1,1,2,2,3], k = 2
- Output: [1,2]

k	v

map.entrySet()

(1,3) (2,2) (3,1)

- Example 2:

- Input: nums = [1], k = 1
- Output: [1]

- Example 3:

- Input: nums = [1,2,1,2,1,2,3,1,3,2], k = 2
- Output: [1,2]

```
int[] topKFrequent(int[] nums, int k) {
```

```
    HashMap<Integer, Integer> map =  
        new HashMap<>();
```

```
    for (num : nums)  
        map.put(num, map.getOrDefault(  
            num, 0) + 1);
```

```
    PriorityQueue<Map.Entry<Integer, Integer>> pq  
        = new PriorityQueue<>((x, y) -> x.getValue()  
            - y.getValue());
```

```
    for (Map.Entry<Integer, Integer> e : map.entrySet()) {  
        pq.offer(e);  
        if (pq.size() > k)  
            pq.poll();  
    }
```

```
    int ans[] = new int[k];  
    for (int i = 0, i < k; i++)  
        ans[i] = pq.poll().getKey();
```

```
    return ans;
```